```
================================================================================
TXT2PDF V1.9.2 - THE SPARTAN PDF GENERATOR
Zero-dependency TXT to PDF converter in pure C11
Now with smart --font auto-detection (CJK) and image DPI awareness
================================================================================

"Turn plain text into structured PDFs without asking for permission.
Now the objects come out folded."

Language:       C11 (pure, no C++ runtime)
Compression:    libdeflate (zlib / FlateDecode), statically embedded,
                enabled by default at level 12
PDF version:    PDF 1.5 by default (PDF 1.4 with --pdf14)
Object streams: enabled by default; disable with --object-streams=disable
Binary:         Static, stripped, musl libc, ~114-134 KB
                (architecture dependent)
Output:         PDF 1.5 by default (PDF 1.4 with --pdf14) with outlines,
                internal links, Base14 fonts, optional embedded Unicode
                TrueType fonts, images with DPI awareness, zlib-compressed
                streams, native object streams, native cross-reference
                streams, and clickable URL links
Status:         v1.9.2 smart --font auto-detection release

================================================================================
QUICK START
================================================================================

- Compile with the provided script (builds libdeflate and txt2pdf):
  ./compile.sh platform or all

- Run with defaults (PDF 1.5 + object streams, Courier font,
  zlib compression level 12):
  ./txt2pdf readme.txt readme.pdf

- Run legacy PDF 1.4 output:
  ./txt2pdf --pdf14 readme.txt readme.pdf

- Disable object streams:
  ./txt2pdf --object-streams=disable readme.txt readme.pdf

- Force object streams explicitly:
  ./txt2pdf --object-streams=generate readme.txt readme.pdf

- Disable compression (classic v1.7.2 stream behavior):
  ./txt2pdf --no-compress readme.txt readme.pdf

- Choose a compression level (1 fastest ... 12 smallest):
  ./txt2pdf --level 9 readme.txt readme.pdf

- Run with Helvetica:
  ./txt2pdf --sans readme.txt readme.pdf

- Run with Times-Roman:
  ./txt2pdf --serif readme.txt readme.pdf

- Run with an embedded Unicode TrueType font:
  ./txt2pdf --font NotoSansJP-Regular.ttf readme.txt readme.pdf

- Force image DPI and center images:
  ./txt2pdf -res 96 --center-images readme.txt readme.pdf
```

- Run with debug diagnostics:
  ./txt2pdf --debug readme.txt readme.pdf

- Version and help:
  ./txt2pdf --version
  ./txt2pdf --help

==
[TOC] TABLE OF CONTENTS
==

1.What is txt2pdf?
2.What is new in v1.9.2?
3.What is new in v1.9.1?
4.What is new in v1.9.0?
5.Fonts and typography
6.Stream compression (libdeflate)
7.Url links
8.Image support
9.Parser rules
10.Table of contents rules
11.Usage and debug mode
12.Minimal working structure
13.Pdf internals
14.Project structure
15.Building and static cookbook
16.Known limitations
17.Troubleshooting
18.Comparison and design philosoph
19.License and credits

==
1. WHAT IS TXT2PDF?
==

txt2pdf is a zero-dependency command-line utility that converts plain text
files into structured PDF 1.5 documents by default. Legacy PDF 1.4 output
is still available with --pdf14.

Unlike Pandoc, wkhtmltopdf, or Python-based converters that require hundreds
of megabytes of runtime dependencies, txt2pdf is a small static binary.
It parses text, calculates page breaks, generates PDF objects, compresses
the streams with zlib (libdeflate), and writes the final binary stream
directly to disk using only the C standard library plus the statically
embedded libdeflate compressor.

Since v1.9.0, txt2pdf can generate native PDF object streams (/ObjStm) and
native cross-reference streams (/XRef), reducing final PDF size without any
external post-processing tool.

Since v1.9.1, txt2pdf reads image DPI metadata (JPEG JFIF density and PNG
pHYs chunks), supports per-image and global centering, and reports accurate
file size statistics.

It is designed for environments where installing dependencies is impossible,
unwanted, or where the network is down.

==
2. WHAT IS NEW IN V1.9.2?
==
Version 1.9.2 makes --font smart: the Unicode TrueType pipeline is

only activated when the document actually contains CJK text.

New behavior:
- --font FILE.ttf now scans the document for real CJK codepoints
  (ideographs, kana, hangul, fullwidth forms).
- Documents without CJK text keep the Base14 font (Courier by
  default, or the one selected with --sans/--serif), so:
  - No font is embedded: PDFs stay minimal.
  - Fixed-width ASCII art and log layouts stay perfectly aligned.
- Documents with CJK text embed the TrueType font exactly as
  before (subsetted, Identity-H, ToUnicode).
- Characters outside WinAnsi that are not CJK (e.g. an isolated
  emoji) are printed as '?' in Base14 mode, as before.

New --debug diagnostics:
- [debug] No CJK chars detected: keeping Base14 font (--font ignored)
- [debug] CJK chars detected, embedding font: <path>

All v1.9.1 features remain unchanged:
- Image DPI awareness (JPEG JFIF, PNG pHYs) and -res N.
- --center-images and [CENTER][IMG]...[/IMG][/CENTER].
- Accurate file size statistics.

All v1.9.0 features also remain unchanged:
- Native PDF 1.5 object streams and cross-reference streams.
- Internal TOC links with codepoint-exact click rectangles.

All v1.8.0 features also remain unchanged:
- zlib stream compression with libdeflate (level 12 by default).
- External URL links (http://, https://, ftp://, www.).
- Multi-architecture builds and deterministic builds.

==
3. WHAT IS NEW IN V1.9.1?
==

Version 1.9.1 is a bug-fix and polish release. No new major features, but
several correctness and usability issues are resolved.

Bug fixes:
- Fixed [CENTER][IMG]...[/IMG][/CENTER] parsing: the centered flag was being
  silently overwritten by the JPEG/PNG passthrough loaders, so centered
  images always appeared left-aligned. The flag is now applied after the
  image is loaded.
- Fixed a critical argument parsing bug where --level was ignored due to a
  duplicated empty else-if block. Passing --level 9 now correctly sets the
  compression level.
- Added an early access() check to validate input file readability before
  processing, providing clearer error messages instead of a generic
  "Cannot read".

Codebase cleanup:
- Removed dead/placeholder code for image re-encoding (-q/--quality,
  --copy-image, --no-copy-image) to maintain a strictly spartan codebase.
  Users should pre-process images externally for resizing or compression.

Console output improvements:
- The final summary now reports the actual file size and the passthrough
  image bytes, so the numbers add up transparently:

  Compression: zlib level 12 (libdeflate)

```
  Streams: 7697 -> 2161 bytes (71.9% saved)
  Images (passthrough, not compressed): 10824 bytes
  Final File size: 13583 bytes

- --debug now prints the centered flag for every detected image.

All v1.9.0 features remain unchanged:
- Native PDF 1.5 output by default.
- Native object streams (/ObjStm) and cross-reference streams (/XRef).
- --pdf15, --pdf14, --object-streams=generate, --object-streams=disable.
- Internal TOC links rendered as plain blue text without underline.
- Codepoint-aware Courier metrics for exact link rectangles.
- Parser accepts glued indexes and mixed-case section titles.

All v1.8.0 features also remain unchanged:
- zlib stream compression using libdeflate.
- External URL support (http://, https://, ftp://, www.).
- Multi-architecture build script (amd64, arm64, riscv64).
- Deterministic builds with SOURCE_DATE_EPOCH and ZERO_AR_DATE.

================================================================================
4. WHAT IS NEW IN V1.9.0?
================================================================================

Version 1.9.0 adds native PDF 1.5 object streams and native cross-reference
streams.

Main changes:
- Native PDF 1.5 output by default.
- Native object streams (/ObjStm) for structural non-stream PDF objects.
- Native cross-reference streams (/XRef) with binary /W [1 4 2] entries.
- Object streams and xref streams are compressed with libdeflate when useful.
- Added --pdf15, --pdf14, --object-streams=generate,
  --object-streams=disable, --objstm and --no-object-streams.

- Default behavior:
  PDF 1.5 + object streams + no linearization.
- Linearization is intentionally not generated. Linearization optimizes
  Fast Web View, not minimum file size.
- Classic PDF 1.4 output remains available with --pdf14.
- Generated PDF 1.5 output passes qpdf --check.
- Internal TOC links are rendered as plain blue text without
  underline; link annotations use an invisible border and
  codepoint-exact clickable rectangles.
- Codepoint-aware Courier metrics: UTF-8 accents count as one
  character, so link rectangles end exactly at the last printed
  letter (trailing spaces, tabs and dots are trimmed).
- Parser: glued indexes ("1.what is txt2pdf?") and mixed-case
  section titles are accepted natively.
- New --debug diagnostics: unparsable TOC lines, TOC entries without
  a matching section, and section headers without a TOC link
  (useful when the Bookmarks and Internal links counts differ).

Example PDF 1.5 impact (Spanish manual from Baresnap):
- txt2pdf v1.8 PDF 1.4:              373965 bytes
- qpdf --object-streams=generate:     291972 bytes
- txt2pdf v1.9 native PDF 1.5:        289417 bytes

In this test, native txt2pdf v1.9 output is slightly smaller than the same
document post-processed with qpdf, while remaining fully self-contained.
```

All v1.8.0 features remain unchanged:
- zlib stream compression using libdeflate (/Filter /FlateDecode).
- Compression enabled by default at level 12.
- --level N to choose the compression level.
- --no-compress to generate uncompressed PDFs.
- Compressed streams: page content streams, ToUnicode CMap and FontFile2.
- Images (JPEG/PNG) are never recompressed.
- External URL support: http://, https://, ftp:// and www. links.
- URLs become clickable /URI link annotations, rendered in blue with
  underline.
- --version / -V and --help / -h.
- Unknown options produce an error instead of being ignored.
- Multi-architecture build script: amd64, arm64, riscv64.
- Deterministic builds with SOURCE_DATE_EPOCH and ZERO_AR_DATE.
- Compression summary printed after each conversion.

All v1.7.2 features also remain unchanged:
- CJK mode with --font FILE.ttf.
- TrueType subsetting.
- JPEG and PNG image passthrough.
- Outlines, internal links, UTF-8 and WinAnsi support.

==
5. FONTS AND TYPOGRAPHY
==

txt2pdf v1.9.2 supports two font pipelines.

A. Base14 fonts (classic mode):
  --mono   Courier (default, classic txt2pdf layout)
  --sans   Helvetica (clean, modern, proportional)
  --serif   Times-Roman (classic documentation look)

Examples:
  ./txt2pdf --mono input.txt output.pdf
  ./txt2pdf --sans input.txt output.pdf
  ./txt2pdf --serif input.txt output.pdf

Important notes:
- Base14 fonts are not embedded into the PDF.
- PDF viewers normally provide built-in substitutes.
- Courier layout remains fixed-width and highly predictable.
- Helvetica and Times-Roman use built-in ASCII metrics for wrapping.
- Accents and non-ASCII characters remain limited to WinAnsiEncoding.

B. Embedded TrueType font (CJK mode):
  --font FILE.ttf

Examples:
  ./txt2pdf --font NotoSansJP-Regular.ttf roadmap.txt roadmap.pdf
  ./txt2pdf --font NotoSansSC-Regular.ttf input.txt output.pdf
  ./txt2pdf --font NotoSansKR-Regular.ttf input.txt output.pdf

Important notes:
- The font must be an individual TrueType .ttf file.
- OpenType/CFF .otf files are not supported yet.
- TrueType collections .ttc are not supported yet.
- When --font is used, it overrides --sans, --serif and --mono.
- Since v1.9.2, --font only activates the TrueType pipeline when the
  document contains real CJK codepoints; otherwise the Base14 font
  is kept (no embedding, ASCII art stays aligned).

- The selected font is used for ALL text (ASCII, Latin, CJK).
- Only the glyphs actually used in the document are embedded.
- The embedded FontFile2 stream is zlib-compressed by default.

==
6. STREAM COMPRESSION (LIBDEFLATE)
==

txt2pdf v1.9.2 compresses PDF streams with zlib using the libdeflate
library, which is compiled directly into the static binary.

What gets compressed:
- Page content streams (the actual text drawing operators).
- The ToUnicode CMap stream (CJK mode).
- The FontFile2 stream (embedded TrueType subset, CJK mode).
- When PDF 1.5 object streams are enabled, the object stream itself and the
  cross-reference stream are also compressed with libdeflate when useful.

What is never recompressed:
- JPEG images (already DCTDecode).
- PNG images (already FlateDecode with PNG predictor).

Options:
  (default)   Compression ON, zlib level 12.
  --level N   Compression ON, level N (1 = fastest, 12 = smallest).
  --compress   Explicit ON. Accepted for scripting; it is the default.
  --no-compress   Compression OFF, classic v1.7.2 stream behavior.

Examples:
  ./txt2pdf input.txt output.pdf
  ./txt2pdf --level 6 input.txt output.pdf
  ./txt2pdf --no-compress input.txt output.pdf

Behavior notes:
- Each stream is checked: if compressing makes it bigger, it is stored
  uncompressed.
- Compressed streams are declared with /Filter /FlateDecode and the
  /Length entry always reflects the stored (compressed) size.
- Level 12 maximizes compression ratio but is slower. For huge documents
  try --level 6 for a good speed/size balance.
- At the end of each run a summary is printed, for example:

  Compression: zlib level 12 (libdeflate)
  Streams: 7697 -> 2161 bytes (71.9% saved)
  Images (passthrough, not compressed): 10824 bytes
  Final File size: 13583 bytes

==
7. URL LINKS
==

External URLs in the text are automatically detected and converted to
clickable links.

Supported URL patterns:
- http://example.com
- https://github.com/johna124
- ftp://files.example.org
- www.example.com (automatically prefixed with https://)

URLs are rendered in blue with underline (HTML-style) and are clickable

in any PDF viewer. Trailing punctuation (. , ; : ! ?) is automatically
stripped from the link.

Example:
  Also http://example.com and www.example.org are working.

All detected URLs become clickable blue underlined links.

Internal TOC links follow a different visual style: they are rendered
in blue WITHOUT underline, keeping the index clean. External URLs
keep the classic blue underline.

Limitations:
- URLs are only linked when they fit entirely inside one visual line.
- URL detection does not support line breaks within the URL.
- URLs inside TOC blocks are not scanned (TOC lines are reserved for
  internal links).

==
8. IMAGE SUPPORT
==

txt2pdf supports standalone block images with DPI awareness.

Supported image tags:
  [IMG]photo.jpg[/IMG]
  [IMG]diagram.jpeg[/IMG]
  [IMG]chart.png[/IMG]
  [CENTER][IMG]logo.jpg[/IMG][/CENTER]

The image line must be the only content on that line.

Good image line:
  [IMG]chart.png[/IMG]

Bad image line:
  See this chart: [IMG]chart.png[/IMG]

Supported formats:
- JPEG: Embedded using PDF native DCTDecode passthrough.
- PNG: Embedded using a minimalist passthrough for simple PNG files.

Supported PNG types:
- 8-bit grayscale
- 8-bit RGB
- 8-bit palette

Unsupported PNG types:
- alpha transparency
- interlaced PNGs
- 16-bit channels
- palette transparency

Image DPI awareness:
- JPEG JFIF density and PNG pHYs chunks are read automatically.
- Images that declare their own DPI are sized correctly without forcing
  a global value.
- Use -res N or --image-dpi N to override the display DPI for all images
  (e.g. 72, 96, 300).
- Default DPI is 96 when the image does not declare one.

Image centering:
- Use --center-images to center all [IMG] blocks horizontally.
- Use [CENTER][IMG]file[/IMG][/CENTER] for per-image centering.
- The per-image tag overrides --center-images when both are used.

Image layout behavior:
- Images are treated as standalone content blocks.
- Aspect ratio is preserved.
- Images wider than the text area are scaled down.
- Images taller than the page are scaled down to fit.
- An image may force a page break if it does not fit.
- Image paths are relative to the current working directory.
- The PDF stores the original compressed image data untouched.
- Reducing source image size reduces final PDF size.

Important:
Images are embedded untouched. If you need to resize or recompress them,
do it on the source file before running txt2pdf.

==========
9. PARSER RULES
==========

The parser creates PDF bookmarks only for lines that are real section
headers.

A line is considered a section header only if all of these conditions are
true:
- It starts with a digit, optionally followed by letters and dots
  (hierarchical IDs such as 2.3, 11.14, 13.1b).
- The title text after the dot contains at least one alphabetic
  character, in any case ("1. WHAT", "1. What" and "1. what" work).
- The space after the numeric dot is optional: "1.what" is valid.
- The line is surrounded by separator lines made mostly of '=' characters.
- The line is not inside a contents block.

Valid section shape:
- separator line
- numbered title (any case)
- separator line

Rejected lines:
- numbered titles without any alphabetic character
- numbered titles used as examples inside normal paragraphs
- numbered titles not surrounded by separator lines
- numbered titles inside the contents block

This makes the parser much safer for manuals that contain examples.

The parser supports hierarchical IDs such as:
- 1
- 2.3
- 2.3.1
- 11.14
- 2.0a
- 13.1b
- 26.5a

==========
10. TABLE OF CONTENTS RULES
==========

Internal links are created only from contents blocks.

A contents block is detected when:
- a line contains the token "[TOC]"
- the next non-blank line is a separator line
- the block ends at the next separator line

Rules for entries:
- entries must be numbered lines (the space after the dot is optional)
- lowercase entries are recommended
- entries are not converted into bookmarks
- entries are linked to the matching section by number or title
- uppercase entries inside the contents block are also ignored as bookmarks

This means the old duplicate-bookmark problem is largely eliminated by the
parser itself.

==
11. USAGE AND DEBUG MODE
==

Basic usage:
  ./txt2pdf input.txt output.pdf

Font selection:
  ./txt2pdf --mono input.txt output.pdf
  ./txt2pdf --sans input.txt output.pdf
  ./txt2pdf --serif input.txt output.pdf
  ./txt2pdf --font FILE.ttf input.txt output.pdf

Compression control:
  ./txt2pdf input.txt output.pdf
  ./txt2pdf --level 6 input.txt output.pdf
  ./txt2pdf --compress input.txt output.pdf
  ./txt2pdf --no-compress input.txt output.pdf

PDF version / object streams:
  ./txt2pdf input.txt output.pdf
  ./txt2pdf --pdf15 input.txt output.pdf
  ./txt2pdf --pdf14 input.txt output.pdf
  ./txt2pdf --object-streams=generate input.txt output.pdf
  ./txt2pdf --object-streams=disable input.txt output.pdf
  ./txt2pdf --objstm input.txt output.pdf
  ./txt2pdf --no-object-streams input.txt output.pdf

Image control:
  ./txt2pdf -res 96 input.txt output.pdf
  ./txt2pdf --image-dpi 300 input.txt output.pdf
  ./txt2pdf --center-images input.txt output.pdf

Information:
  ./txt2pdf --help
  ./txt2pdf --version

Debug usage:
  ./txt2pdf --debug input.txt output.pdf

Combined usage:
  ./txt2pdf --debug --sans input.txt output.pdf
  ./txt2pdf --debug --level 9 input.txt output.pdf

```
  ./txt2pdf --debug --font NotoSansJP-Regular.ttf input.txt output.pdf
  ./txt2pdf --debug -res 96 --center-images input.txt output.pdf

Debug mode prints diagnostics to stderr.

It reports:
- detected encoding
- selected font
- the font auto-detection decision (CJK pipeline vs Base14)
- compression mode and level
- PDF version and object-stream mode
- lines marked as part of a contents block
- detected section headers
- skipped uppercase numbered lines
- detected internal links
- TOC lines that cannot be parsed as numbered entries
- TOC entries without a matching section
- section headers without any TOC link pointing to them
- detected images and image placements (including centered flag)
- CJK glyph count and subset size (in CJK mode)

Use debug mode when:
- the Bookmarks and Internal links counts do not match
- a bookmark is missing
- too many bookmarks appear
- a contents entry does not link anywhere
- an image does not appear or is not centered
- CJK characters render as boxes or ?
- you are adapting a new readme to the Spartan format

================================================================================
12. MINIMAL WORKING STRUCTURE
================================================================================

A minimal compatible document needs three things:
- a contents block starting with the [TOC] token
- lowercase or mixed-case contents entries
- uppercase section headers surrounded by separator lines

Recommended structure:
- write the contents title line containing [TOC]
- write a separator line
- write entries such as "1. introduction" and "2. conclusion"
- write a separator line
- for each section, write a separator line
- write the uppercase header, for example "1. INTRODUCTION"
- write another separator line
- write the section body

Important:
Do not wrap example headers with separator lines unless you want them to
become real bookmarks.

================================================================================
13. PDF INTERNALS
================================================================================

txt2pdf generates a valid PDF 1.5 structure by default. Legacy PDF 1.4
output can be forced with --pdf14.

Catalog and Pages:
```

Calculates A4 dimensions, applies margins, wraps text, and places content
using a line-based layout.

Fonts:
Generates a Base14 Type1 font object. The default is Courier. The --sans
and --serif options switch to Helvetica or Times-Roman.

In CJK mode, it generates a Type0 font tree with a CIDFontType2 descendant,
an embedded FontFile2 stream (subsetted TTF), and a ToUnicode CMap.

Streams and Compression:
When compression is enabled (the default), page content streams, the
ToUnicode CMap and the FontFile2 stream are compressed with libdeflate in
zlib format and declared with /Filter /FlateDecode. The /Length entry
always reflects the stored compressed size. Streams that would grow when
compressed are stored uncompressed instead. Image streams are never
recompressed.

Object Streams:
When object streams are enabled, structural non-stream objects are grouped
into a PDF /ObjStm object and compressed with libdeflate. Stream objects
such as page content streams, image XObjects, FontFile2 and ToUnicode CMap
remain separate objects, as required by the PDF specification.

Images:
Loads image metadata (including DPI from JFIF and pHYs), reserves vertical
layout space, writes image XObject streams, and references them from page
resources. Centered images are positioned at the horizontal center of the
text area.

URL Links:
Scans text lines for http://, https://, ftp:// and www. patterns. Detected
URLs are rendered in blue with underline in the content stream and
converted to /URI link annotations.

Encoding and Smart Decoder:
Auto-detects UTF-8 vs Latin-1. If UTF-8 is detected, a smart decoder
translates supported multi-byte sequences to WinAnsiEncoding.

In CJK mode, UTF-8 codepoints are mapped directly to glyph IDs from the
embedded TrueType font, and the content stream uses hexadecimal notation
with Identity-H encoding.

Outlines:
Generates a linked list of PDF Outline objects pointing to the exact XYZ
coordinates of each uppercase header. In CJK mode, outline titles use
UTF-16BE hex encoding so they display CJK characters correctly.

Annotations:
Scans contents blocks and injects link annotations for each entry matching
a real section. URL links are injected as /URI annotations.

Cross-Reference:
In PDF 1.5 object-stream mode, txt2pdf writes a binary /XRef stream using
/W [1 4 2]. Entries can reference normal objects by file offset, or objects
stored inside an object stream by object-stream number and index.

In PDF 1.4 mode, txt2pdf writes the classic xref table and trailer.

Offsets are computed after compression, so they always point to the exact
start of each object.

==
14. PROJECT STRUCTURE
==

The txt2pdf codebase follows a single-file architecture plus one vendored
compression library.

Typical layout:
- txt2pdf.c
- compile.sh
- readme.txt
- README.md
- libdeflate/   (vendored zlib/deflate compressor sources)

The main C file contains:
- argument parsing
- encoding detection
- text parsing
- header detection
- contents detection
- image loading (with DPI awareness)
- font selection (Base14 and TrueType)
- TrueType subsetting
- stream compression (libdeflate)
- URL detection
- page break calculation
- PDF object generation
- PDF 1.5 object-stream generation
- binary stream writing

The tool is intentionally small and auditable.

==
15. BUILDING AND STATIC COOKBOOK
==

Provided build script:
  ./compile.sh

What compile.sh does:
- Builds libdeflate as a static library.
- If libdeflate has a Makefile and make is available, it uses it.
- Otherwise it compiles the libdeflate/lib sources directly.
- CMake is NOT required.
- Verifies the library exports the needed symbols.
- Links txt2pdf against it as a fully static binary.
- Auto-downloads musl.cc toolchains if missing (for cross-compilation).
- Supports multi-architecture builds: amd64, arm64, riscv64.
- Uses deterministic build settings (SOURCE_DATE_EPOCH, ZERO_AR_DATE).

Build flags used to keep the binary small:
  -Os -ffunction-sections -fdata-sections
  -fno-unwind-tables -fno-asynchronous-unwind-tables
  -static -Wl,--gc-sections -Wl,--build-id=none -s

Expected result:
  wc -c txt2pdf-amd64
  132848 bytes (129 KB)

  wc -c txt2pdf-arm64

```
  137352 bytes (134 KB)

  wc -c txt2pdf-riscv64
  116792 bytes (114 KB)

RISC-V is usually the smallest thanks to RVC (compressed 16-bit
instructions).

Multi-architecture builds:
  ./compile.sh              (build for host architecture)
  ./compile.sh amd64        (build for x86_64)
  ./compile.sh arm64        (build for aarch64)
  ./compile.sh riscv64      (build for RISC-V 64-bit)
  ./compile.sh all          (build all three at once)

Manual build (once libdeflate/libdeflate.a exists):
  musl-gcc -Os -Wall -Wextra -std=c11 -static -Ilibdeflate \
    -o txt2pdf txt2pdf.c libdeflate/libdeflate.a
  strip txt2pdf

Why musl?
glibc static binaries are often much larger due to NSS and locale data.
musl provides a clean, minimal libc that results in a tiny, fully
self-contained executable.

Verify static build:
  file txt2pdf
  ldd txt2pdf

Expected:
- file reports statically linked
- ldd says it is not a dynamic executable

Optional extra squeeze (if UPX is installed):
  upx --best --lzma txt2pdf

Force a libdeflate rebuild:
  rm -rf libdeflate/build libdeflate/libdeflate.a
  ./compile.sh

================================================================================
16. KNOWN LIMITATIONS
================================================================================

General:
- Object streams require PDF 1.5; use --pdf14 for legacy viewers.
- No linearization (Fast Web View) is generated.
- No custom page sizes.
- No custom margins.
- No multi-column layout.
- No inline images.
- No image captions.
- No repeated header/footer images.

Images:
- PNG support is limited to simple 8-bit non-interlaced images.
- PNG alpha transparency is not supported.
- Interlaced PNGs are not supported.
- Images are embedded untouched; no re-encoding is performed.
- To resize or recompress images, do it on the source file before
  running txt2pdf.
```

URLs:
- URLs are only linked when they fit entirely inside one visual line.
- URL detection does not support line breaks within the URL.
- URLs inside TOC blocks are not scanned.

Base14 mode:
- WinAnsiEncoding only.
- Characters outside WinAnsi are replaced with '?'.
- Base14 fonts are not embedded.

CJK mode:
- Requires an individual TrueType .ttf file.
- Since v1.9.2, --font is auto-detected: documents without CJK
  codepoints keep the Base14 font (nothing is embedded).
- OpenType/CFF .otf files are not supported yet.
- TrueType collections .ttc are not supported yet.
- No automatic per-script font fallback.
- Scripts requiring shaping (Arabic, Indic, Thai) are not supported.

Compression:
- Only text-bearing streams are compressed (content, ToUnicode, FontFile2).
- Image streams are never recompressed.
- Object streams and xref streams are only used in PDF 1.5 object-stream
  mode.
- Level 12 is the slowest level; huge documents may prefer --level 6.
- libdeflate sources must be present in libdeflate/ at build time.

Parser:
- Section headers must be fenced by separator lines.
- Internal links are only generated from contents blocks.

==
17. TROUBLESHOOTING
==

--font seems to be ignored:
- Since v1.9.2, --font only activates when the document contains
  real CJK codepoints. Run with --debug to see the decision.
- Isolated emoji/symbols do not activate the TrueType pipeline;
  they are printed as '?' in Base14 mode.

Bookmarks and Internal links counts differ:
- A section header has no entry in the contents block (or vice versa).
- The entry number or title does not match the header exactly.
- Run with --debug: it prints the orphan TOC entries and the section
  headers without a TOC link.

Missing bookmarks:
- The header contains lowercase letters.
- The header is not surrounded by separator lines.
- The header is accidentally inside a contents block.

Duplicate bookmarks:
- Old documents may have uppercase contents entries.
- The updated parser ignores contents entries as sections.
- If duplicates persist, run with --debug and inspect skipped lines.

Missing links:
- The entry is not inside a detected contents block.
- The entry number does not match any section.

- The entry title does not match any section title.

Extra bookmarks:
- An example header is surrounded by separator lines.
- Remove the separators around examples or rewrite the example line.

URLs not detected:
- The URL contains line breaks (must fit on one visual line).
- The URL is inside a TOC block (TOC lines are not scanned for URLs).
- Trailing punctuation is stripped automatically.

Garbled characters in Base14 mode:
- Ensure your text editor saves the file in UTF-8.
- Run with --debug to verify the detected encoding.
- Characters outside WinAnsiEncoding cannot be rendered.
- Use --font FILE.ttf for full Unicode support.

CJK characters appear as boxes or '?' in CJK mode:
- The .ttf file does not contain the requested glyphs.
- Use a font with broader coverage (e.g. Noto Sans SC for Simplified
  Chinese, Noto Sans TC for Traditional Chinese, Noto Sans JP for
  Japanese, Noto Sans KR for Korean).
- Check that the file is a real .ttf, not .otf or .ttc.

Cannot load TTF font error:
- The file is not in the current directory. Use absolute path.
- The file is .otf (CFF) instead of .ttf (TrueType).
- The file is a .ttc collection (not supported yet).
- The file is corrupted or incomplete.

Image does not appear:
- The file path is wrong.
- The image line contains extra text beside the tag.
- The image format is not supported.
- The PNG uses alpha, interlacing, or 16-bit channels.

Image is not centered:
- Use --center-images to center all images.
- Use [CENTER][IMG]file[/IMG][/CENTER] for per-image centering.
- The per-image tag overrides --center-images when both are used.

Image is too large or too small:
- Use -res N to force a specific DPI (e.g. -res 72 for larger images,
  -res 300 for smaller images).
- If the image declares its own DPI (JFIF or pHYs), it is used
  automatically.
- High-resolution images are scaled visually but still embedded at
  full resolution.

PDF is not smaller with compression:
- The document is dominated by images; only text streams are compressed.
- Check the final summary lines printed by txt2pdf.
- Verify you are not passing --no-compress.

--level option is ignored:
- Make sure you are using v1.9.1 or later.
- Earlier versions had a duplicated empty else-if block that captured
  --level before the real handler.

libdeflate build problems:
- compile.sh does not use CMake; it is not required.

- If linking reports missing symbols, delete libdeflate/build and
  libdeflate/libdeflate.a and run compile.sh again.
- Make sure the libdeflate sources are complete (fresh clone or release
  tarball); the library needs all files under libdeflate/lib.

Font does not look as expected:
- Use --mono for the classic Courier layout.
- Use --sans for Helvetica.
- Use --serif for Times-Roman.
- Use --font FILE.ttf for Unicode CJK layout.
- Some viewers substitute Base14 fonts with similar fonts.

Debug output is the fastest way to diagnose the problem.

18. COMPARISON AND DESIGN PHILOSOPHY

If you just need to generate a structured PDF from raw text, code, or logs,
using massive office suites or complex pipelines is like using a rocket
launcher to swat a fly.

How txt2pdf compares to other tools:

Feature	txt2pdf	qpdf (post)	enscript/a2ps	pandoc
Primary Purpose	TXT to PDF	PDF optimize	TXT to PS/PDF	Universal
Binary Size	~114-137 KB	~2-5 MB	~500 KB + GS	~50-200 MB+
Runtime Dependencies	NONE	libqpdf	Ghostscript	LaTeX/WebKit
Execution Speed	Blazing fast	Fast (2nd pass)	Moderate	Slow startup
PDF 1.5 Object Streams	YES (native)	YES (extra)	NO	Varies
External URL Links	YES	N/A	NO	YES (via MD)
Internal TOC Links	YES	N/A	NO	YES (via MD)
Deterministic Builds	YES	NO	NO	NO
Target Audience	Hackers/Embedded	PDF devs	Legacy Unix	Academics

The qpdf Advantage:
Historically, minimizing PDF structural overhead required a two-step pipeline:

```
./txt2pdf input.txt temp.pdf
qpdf --object-streams=generate temp.pdf final.pdf
rm temp.pdf
```

With v1.9.0+, txt2pdf generates native PDF 1.5 object streams (/ObjStm) and
cross-reference streams (/XRef) directly inside the static binary. You get
the exact same structural size benefits as qpdf, but in a single,
deterministic pass, with zero external dependencies and no temporary files.

The Spartan Advantage:
Modern software development has forgotten how to build small tools. We wrap
simple text transformations in Electron apps, Node.js runtimes, and Docker
containers.

txt2pdf is a rejection of that bloat. It proves that a fully functional,
structured document generator can fit in a tiny static binary, written in
pure C11.

It natively parses your raw text stream, automatically detects document
hierarchy, and embeds cross-linked internal navigation and PDF outlines out
of the box. It is designed to run inside ultra-low-power RISC-V/ARM64
dev-boards, minimal CI/CD containers, or bare-metal environments where
modern runtimes simply choke.

With v1.7.2, it gained subsetted TrueType embedding for real Unicode CJK
text.
With v1.8.0, it squeezed every text stream through libdeflate at
level 12 and turned every URL into a clickable link.

With v1.9.0, it eliminated the need for qpdf by generating native PDF 1.5 object streams
directly.

And with v1.9.1, it added image DPI awareness, per-image
centering, accurate file size reporting, and a cleaner, strictly spartan
codebase.

==========
19. LICENSE AND CREDITS
==========

Source code : GPLv3
Manual   : GFDL v1.3
libdeflate : MIT license, Copyright 2016 Eric Biggers

txt2pdf was written as a demonstration of minimalist software design.
Inspired by the Unix philosophy: do one thing well.

Built with minimalism, on a potato, for potatoes.

https://github.com/johna124/txt2pdf
https://www.gnu.org/licenses/gpl-3.0.txt
https://www.gnu.org/licenses/fdl-1.3.txt

==========
When the cloud goes down, and the package managers fail to resolve,
the Spartan tool remains.

"Code is neither created nor destroyed; it is distilled."

END OF DOCUMENTATION
==========